CAR 파일 처리

개요

CAR(Content Addressable aRchive) 파일은 IPFS에서 사용하는 아카이브 형식으로, 여러 블록을 하나의 파일로 묶어서 저장하고 전송할 수 있게 해줍니다. Boxo의 CAR 처리 기능은 신뢰할 수 있는 데이터 전송, 효율적인 아카이브 생성, 그리고 검증 가능한 콘텐츠 배포를 지원합니다.

주요 개념

CAR 파일의 특징

1. **콘텐츠 주소 지정**: 각 블록이 해시로 식별됨
2. **자체 검증**: 파일 내용의 무결성을 자체적으로 검증 가능
3. **부분 전송**: 필요한 부분만 선택적으로 전송 가능
4. **순서 보장**: 블록의 순서를 지정하여 효율적인 탐색 지원

CAR 파일 구성 요소

- **헤더**: CAR 버전과 루트 CID 정보
- **블록들**: 실제 데이터 블록들의 연속
- **인덱스**: 블록 위치 정보 (선택적)

기본 사용법

CAR Fetcher 설정

```go
package main


import (
"context"
"log"
"net/http"

"github.com/ipfs/boxo/gateway"
"github.com/ipfs/boxo/path"
)

func  main() {
ctx := context.Background()
// 원격 CAR Fetcher 생성
gatewayURLs := []string{
"https://ipfs.io",
"https://dweb.link",
}
httpClient := &http.Client{
Timeout: 30 * time.Second,
}
carFetcher, err := gateway.NewRemoteCarFetcher(gatewayURLs, httpClient)
```

```
if err != nil {
log.Fatal(err)
}
// CAR 파일 가져오기
immutablePath := path.FromCid(cid.MustParse("QmYourContent..."))
params := gateway.CarParams{
Scope: gateway.DagScopeAll,
}
err = carFetcher.Fetch(ctx, immutablePath, params, func(p path.ImmutablePath, re
log.Printf("CAR 데이터 수신: %s", p)
// CAR 데이터 처리
return processCarData(reader)
})
if err != nil {
log.Fatal(err)
}
}


func processCarData(reader io.Reader) error {
// CAR 데이터를 처리하는 로직
data, err := io.ReadAll(reader)
if err != nil {
return err
```

```
}

log.Printf("CAR 파일 크기: %d 바이트", len(data))

return nil

}
```

CAR Backend 사용

```
// CAR Backend를 사용한 IPFS 게이트웨이 구성

func setupCarBackend() {

gatewayURLs := []string{"https://ipfs.io"}

httpClient := &http.Client{}

// CAR Backend 생성

backend, err := gateway.NewRemoteCarBackend(gatewayURLs, httpClient)

if err != nil {

log.Fatal(err)

}

ctx := context.Background()

immutablePath := path.FromCid(cid.MustParse("QmYourContent..."))

// 파일 가져오기

md, response, err := backend.Get(ctx, immutablePath)

if err != nil {

log.Fatal(err)

}
```

```
defer response.Close()

log.Printf("경로 메타데이터: %+v", md)

// 응답 데이터 읽기

data, err := io.ReadAll(response)

if err != nil {

log.Fatal(err)

}

log.Printf("가져온 데이터 크기: %d 바이트", len(data))

}
```

CAR 파일 생성과 읽기

CAR 파일 생성

```
// DAG에서 CAR 파일 생성

func createCarFile(ctx context.Context, rootCid cid.Cid, dagService ipld.DAGSer

// CAR 라이브러리 사용 (go-car 패키지)

// 실제 구현에서는 go-car 라이브러리를 import해야 함

// 루트 CID 설정

roots := []cid.Cid{rootCid}

// CAR 헤더 작성

header := car.CarHeader{

Roots: roots,
```

```
Version: 1,
}
// 헤더를 출력 스트림에 작성
err := car.WriteHeader(&header, output)
if err != nil {
return fmt.Errorf("CAR 헤더 작성 실패: %w", err)
}
// DAG 탐색하여 모든 블록 수집
visited := cid.NewSet()
var writeBlocks func(c cid.Cid) error
writeBlocks = func(c cid.Cid) error {
if visited.Has(c) {
return nil
}
visited.Add(c)
// 블록 가져오기
block, err := dagService.Get(ctx, c)
if err != nil {
return err
}
// CAR 형식으로 블록 작성
err = car.WriteBlock(output, block)
if err != nil {
```

```
return err
}
// 링크된 블록들 재귀적으로 처리
node, err := dagService.Get(ctx, c)
if err != nil {
return err
}
for _, link := range node.Links() {
err = writeBlocks(link.Cid)
if err != nil {
return err
}
}
return nil
}
return writeBlocks(rootCid)
}
```

CAR 파일 읽기

```
// CAR 파일에서 블록 읽기
func readCarFile(ctx context.Context, carReader io.Reader) error {
```

```
// CAR 헤더 읽기
header, err := car.ReadHeader(carReader)
if err != nil {
return fmt.Errorf("CAR 헤더 읽기 실패: %w", err)
}
log.Printf("CAR 버전: %d", header.Version)
log.Printf("루트 CID 개수: %d", len(header.Roots))
for i, root := range header.Roots {
log.Printf("루트 CID %d: %s", i, root)
}
// 블록들 순차적으로 읽기
blockCount := 0
for {
block, err := car.ReadBlock(carReader)
if err == io.EOF {
break
}
if err != nil {
return fmt.Errorf("블록 읽기 실패: %w", err)
}
blockCount++
log.Printf("블록 %d: CID=%s, 크기=%d",
blockCount, block.Cid(), len(block.RawData()))
```

```
// 블록 데이터 처리

err = processBlock(block)

if  err != nil {

log.Printf("블록 처리 오류: %v", err)

}

}

log.Printf("총 %d개 블록 처리 완료", blockCount)

return  nil

}



func  processBlock(block blocks.Block) error {

// 블록별 처리 로직

data := block.RawData()

// 예시: 텍스트 데이터인지 확인

if  len(data) > 0 && data[0] < 128 {

log.Printf("텍스트 블록 내용: %s", string(data[:min(50, len(data))]))

}

return  nil

}
```

부분 CAR 요청

범위 지정 요청

```
// 특정 바이트 범위만 요청
func fetchCarWithRange(ctx context.Context, fetcher gateway.CarFetcher, targetP
// 처음 1KB만 요청
rangeTo := int64(1023)
params := gateway.CarParams{
Scope: gateway.DagScopeEntity,
Range: &gateway.DagByteRange{
From: 0,
To: &rangeTo,
},
}
err := fetcher.Fetch(ctx, targetPath, params, func(p path.ImmutablePath, reader
log.Printf("부분 CAR 데이터 수신: %s", p)
// 부분 데이터 처리
data, err := io.ReadAll(reader)
if err != nil {
return err
}
log.Printf("수신된 부분 데이터 크기: %d 바이트", len(data))
return nil
})
return err
```

```
}


// 파일 끝에서부터 범위 지정
func fetchCarFromEnd(ctx context.Context, fetcher gateway.CarFetcher, targetPat
// 마지막 500바이트 요청
params := gateway.CarParams{
Scope: gateway.DagScopeEntity,
Range: &gateway.DagByteRange{
From: -500, // 음수는 파일 끝에서부터
},
}
return fetcher.Fetch(ctx, targetPath, params, func(p path.ImmutablePath, reader
data, err := io.ReadAll(reader)
if err != nil {
return err
}
log.Printf("파일 끝 부분 데이터: %d 바이트", len(data))
return nil
})
}
```

스코프별 요청

```
// 다양한 스코프로 CAR 요청
func fetchWithDifferentScopes(ctx context.Context, fetcher gateway.CarFetcher,
scopes := []gateway.DagScope{
gateway.DagScopeBlock, // 단일 블록만
gateway.DagScopeEntity, // 엔티티 전체
gateway.DagScopeAll, // 모든 연결된 블록
}
for _, scope := range scopes {
log.Printf("스코프 %s로 요청 중...", scope)
params := gateway.CarParams{Scope: scope}
err := fetcher.Fetch(ctx, targetPath, params, func(p path.ImmutablePath, reader
data, err := io.ReadAll(reader)
if err != nil {
return err
}
log.Printf("스코프 %s: %d 바이트 수신", scope, len(data))
return nil
})
if err != nil {
log.Printf("스코프 %s 요청 실패: %v", scope, err)
}
}
```

```
return nil
}
```

재시도 및 오류 처리

재시도 Fetcher 사용

```
// 재시도 기능이 있는 CAR Fetcher 설정
func setupRetryCarFetcher() gateway.CarFetcher {
// 기본 원격 Fetcher 생성
gatewayURLs := []string{
"https://ipfs.io",
"https://dweb.link",
"https://cloudflare-ipfs.com",
}
baseFetcher, err := gateway.NewRemoteCarFetcher(gatewayURLs, nil)
if err != nil {
log.Fatal(err)
}
// 재시도 Fetcher로 래핑 (최대 3회 재시도)
retryFetcher, err := gateway.NewRetryCarFetcher(baseFetcher, 3)
if err != nil {
log.Fatal(err)
```

```
}

return retryFetcher

}



// 견고한 CAR 페칭

func robustCarFetch(ctx context.Context, fetcher gateway.CarFetcher, targetPath

params := gateway.CarParams{Scope: gateway.DagScopeAll}

// 타임아웃 설정

ctx, cancel := context.WithTimeout(ctx, 60*time.Second)

defer cancel()

var lastError error

maxRetries := 3

for attempt := 1; attempt <= maxRetries; attempt++ {

log.Printf("CAR 페칭 시도 %d/%d", attempt, maxRetries)

err := fetcher.Fetch(ctx, targetPath, params, func(p path.ImmutablePath, reader

// 데이터 검증을 포함한 처리

return validateAndProcessCarData(reader)

})

if err == nil {

log.Println("CAR 페칭 성공")

return nil

}

lastError = err
```

```
log.Printf("시도 %d 실패: %v", attempt, err)

// 부분 응답 오류인 경우 즉시 재시도

if isPartialResponseError(err) {

log.Println("부분 응답 오류 - 즉시 재시도")

continue

}

// 일반 오류인 경우 대기 후 재시도

if attempt < maxRetries {

waitTime := time.Duration(attempt) * time.Second

log.Printf("%v 후 재시도", waitTime)

select {

case <-time.After(waitTime):

case <-ctx.Done():

return ctx.Err()

}

}

}

return fmt.Errorf("최대 재시도 횟수 초과: %w", lastError)

}



func validateAndProcessCarData(reader io.Reader) error {

// 데이터 무결성 검증

data, err := io.ReadAll(reader)
```

```
if err != nil {
return fmt.Errorf("데이터 읽기 실패: %w", err)
}
if len(data) == 0 {
return fmt.Errorf("빈 CAR 데이터")
}
// CAR 헤더 검증
carReader := bytes.NewReader(data)
_, err = car.ReadHeader(carReader)
if err != nil {
return fmt.Errorf("유효하지 않은 CAR 헤더: %w", err)
}
log.Printf("유효한 CAR 데이터 검증 완료: %d 바이트", len(data))
return nil
}


func isPartialResponseError(err error) bool {
var partialErr gateway.ErrPartialResponse
return errors.As(err, &partialErr)
}
```

아카이브 처리 최적화

스트리밍 처리

```
// 대용량 CAR 파일의 스트리밍 처리
func streamProcessCarFile(ctx context.Context, carReader io.Reader) error {
// 헤더 읽기
header, err := car.ReadHeader(carReader)
if err != nil {
return err
}
log.Printf("스트리밍 처리 시작: %d개 루트", len(header.Roots))
// 블록 처리 통계
stats := &carProcessingStats{
startTime: time.Now(),
}
// 블록별 스트리밍 처리
for {
select {
case <-ctx.Done():
return ctx.Err()
default:
}
block, err := car.ReadBlock(carReader)
if err == io.EOF {
```

```
break
}
if err != nil {
return fmt.Errorf("블록 읽기 실패: %w", err)
}
// 블록 처리
err = processBlockStreaming(block, stats)
if err != nil {
log.Printf("블록 처리 오류 (계속 진행): %v", err)
}
// 진행 상황 보고
if stats.processedBlocks%1000 == 0 {
stats.reportProgress()
}
}
stats.reportFinal()
return nil
}


type carProcessingStats struct {
startTime time.Time
processedBlocks int
totalBytes int64
```

```
errorCount int
}

func (s *carProcessingStats) reportProgress() {
elapsed := time.Since(s.startTime)
rate := float64(s.processedBlocks) / elapsed.Seconds()
log.Printf("진행 상황: %d 블록 처리됨 (%.2f 블록/초, %d 오류)",
s.processedBlocks, rate, s.errorCount)
}

func (s *carProcessingStats) reportFinal() {
elapsed := time.Since(s.startTime)
avgRate := float64(s.processedBlocks) / elapsed.Seconds()
log.Printf("처리 완료: %d 블록, %d 바이트, %v 소요 (평균 %.2f 블록/초)",
s.processedBlocks, s.totalBytes, elapsed, avgRate)
}

func processBlockStreaming(block blocks.Block, stats *carProcessingStats) error
stats.processedBlocks++
stats.totalBytes += int64(len(block.RawData()))
// 블록 타입별 처리
switch block.Cid().Type() {
case uint64(multicodec.DagPb):
```

```
return processDagPbBlock(block)
case uint64(multicodec.Raw):
return processRawBlock(block)
default:
log.Printf("알 수 없는 블록 타입: %d", block.Cid().Type())
}
return nil
}

func processDagPbBlock(block blocks.Block) error {
// DAG-PB 블록 처리 로직
log.Printf("DAG-PB 블록 처리: %s", block.Cid())
return nil
}

func processRawBlock(block blocks.Block) error {
// Raw 블록 처리 로직
data := block.RawData()
log.Printf("Raw 블록 처리: %s (%d 바이트)", block.Cid(), len(data))
return nil
}
```

병렬 처리

```
// 병렬 CAR 블록 처리
func parallelProcessCarFile(ctx context.Context, carReader io.Reader, workerCou
// 헤더 읽기
header, err := car.ReadHeader(carReader)
if err != nil {
return err
}
// 블록 채널 생성
blockCh := make(chan blocks.Block, 100)
resultCh := make(chan blockProcessResult, 100)
// 워커 고루틴 시작
var wg sync.WaitGroup
for i := 0; i < workerCount; i++ {
wg.Add(1)
go func(workerID int) {
defer wg.Done()
carBlockWorker(ctx, workerID, blockCh, resultCh)
}(i)
}
// 결과 수집 고루틴
go func() {
wg.Wait()
```

```
close(resultCh)

}()

// 블록 읽기 고루틴

go func() {

defer close(blockCh)

for {

block, err := car.ReadBlock(carReader)

if err == io.EOF {

break

}

if err != nil {

log.Printf("블록 읽기 오류: %v", err)

break

}

select {

case blockCh <- block:

case <-ctx.Done():

return

}

}

}()

// 결과 처리

processedCount := 0
```

```
errorCount := 0
for result := range resultCh {
processedCount++
if result.err != nil {
errorCount++
log.Printf("블록 처리 오류: %v", result.err)
}
if processedCount%500 == 0 {
log.Printf("병렬 처리 진행: %d 블록 완료 (%d 오류)",
processedCount, errorCount)
}
}
log.Printf("병렬 처리 완료: %d 블록, %d 오류", processedCount, errorCount)
return nil
}


type blockProcessResult struct {
cid cid.Cid
err error
}


func carBlockWorker(ctx context.Context, workerID int, blockCh <-chan blocks.B
log.Printf("워커 %d 시작", workerID)
```

```
for {
select {
case <-ctx.Done():
return
case  block, ok := <-blockCh:
if !ok {
return
}
// 블록 처리
err := processBlockWithValidation(block)
select {
case  resultCh <- blockProcessResult{cid: block.Cid(), err: err}:
case <-ctx.Done():
return
}
}
}
}


func  processBlockWithValidation(block blocks.Block) error {
// CID 검증
expectedCid, err := block.Cid().Prefix().Sum(block.RawData())
if  err != nil {
```

```
return fmt.Errorf("CID 계산 실패: %w", err)
}
if !expectedCid.Equals(block.Cid()) {
return fmt.Errorf("CID 불일치: 예상 %s, 실제 %s", expectedCid, block.Cid())
}
// 블록 데이터 처리
return processBlockData(block.RawData())
}


func processBlockData(data []byte) error {
// 실제 블록 데이터 처리 로직
if len(data) == 0 {
return fmt.Errorf("빈 블록 데이터")
}
// 예시: 데이터 압축률 계산
compressed := compress(data)
ratio := float64(len(compressed)) / float64(len(data))
if ratio < 0.5 {
log.Printf("높은 압축률 블록: %.2f%%", ratio*100)
}
return nil
}
```

```go
func compress(data []byte) []byte {
// 간단한 압축 시뮬레이션
return data[:len(data)/2]
}
```

모범 사례

1. 효율적인 CAR 생성

```go
// 최적화된 CAR 파일 생성
func createOptimizedCarFile(ctx context.Context, rootCid cid.Cid, dagService ip
// 블록 순서 최적화를 위한 DFS 탐색
visitOrder := make([]cid.Cid, 0)
visited := cid.NewSet()
var collectBlocks func(c cid.Cid) error
collectBlocks = func(c cid.Cid) error {
if visited.Has(c) {
return nil
}
visited.Add(c)
visitOrder = append(visitOrder, c)
// 자식 블록들 먼저 수집 (DFS)
node, err := dagService.Get(ctx, c)
```

```
if err != nil {
return err
}
for _, link := range node.Links() {
err = collectBlocks(link.Cid)
if err != nil {
return err
}
}
return nil
}
// 블록 수집
err := collectBlocks(rootCid)
if err != nil {
return err
}
// CAR 헤더 작성
header := car.CarHeader{
Roots: []cid.Cid{rootCid},
Version: 1,
}
err = car.WriteHeader(&header, output)
if err != nil {
```

```
return err
}
// 최적화된 순서로 블록 작성
for _, c := range visitOrder {
block, err := dagService.Get(ctx, c)
if err != nil {
return err
}
err = car.WriteBlock(output, block)
if err != nil {
return err
}
}
log.Printf("최적화된 CAR 파일 생성 완료: %d 블록", len(visitOrder))
return nil
}
```

2. 메모리 효율적인 처리

```
// 메모리 사용량을 제한한 CAR 처리
func memoryEfficientCarProcessing(ctx context.Context, carReader io.Reader, max
maxMemoryBytes := int64(maxMemoryMB * 1024 * 1024)
currentMemory := int64(0)
```

```
// 헤더 읽기
header, err := car.ReadHeader(carReader)
if err != nil {
return err
}
blockBuffer := make([]blocks.Block, 0, 100)
for {
block, err := car.ReadBlock(carReader)
if err == io.EOF {
// 남은 블록들 처리
if len(blockBuffer) > 0 {
err = processBatchBlocks(blockBuffer)
if err != nil {
return err
}
}
break
}
if err != nil {
return err
}
blockSize := int64(len(block.RawData()))
// 메모리 한계 확인
```

```
if currentMemory+blockSize > maxMemoryBytes {
// 버퍼된 블록들 처리
err = processBatchBlocks(blockBuffer)
if err != nil {
return err
}
// 버퍼 초기화
blockBuffer = blockBuffer[:0]
currentMemory = 0
// GC 실행
runtime.GC()
}
blockBuffer = append(blockBuffer, block)
currentMemory += blockSize
}
return nil
}


func processBatchBlocks(blocks []blocks.Block) error {
log.Printf("배치 처리: %d 블록", len(blocks))
for _, block := range blocks {
err := processBlockData(block.RawData())
if err != nil {
```

```
log.Printf("블록 처리 오류: %v", err)
}
}
return nil
}
```

3. 오류 복구

```
// 손상된 CAR 파일 복구 시도
func recoverCorruptedCarFile(ctx context.Context, carReader io.Reader, output i
log.Println("손상된 CAR 파일 복구 시도 중...")
// 헤더 읽기 시도
header, err := car.ReadHeader(carReader)
if err != nil {
log.Printf("헤더 읽기 실패, 기본 헤더 사용: %v", err)
// 기본 헤더 생성
header = &car.CarHeader{
Roots: []cid.Cid{}, // 빈 루트
Version: 1,
}
}
// 복구된 헤더 작성
```

```
err = car.WriteHeader(header, output)

if err != nil {

return err

}

recoveredBlocks := 0

corruptedBlocks := 0

// 블록별 복구 시도

for {

block, err := car.ReadBlock(carReader)

if err == io.EOF {

break

}

if err != nil {

corruptedBlocks++

log.Printf("손상된 블록 건너뛰기: %v", err)

// 다음 유효한 블록 찾기 시도

err = skipToNextValidBlock(carReader)

if err != nil {

log.Printf("다음 블록 찾기 실패: %v", err)

break

}

continue

}
```

```
// 블록 검증
if validateBlock(block) {
err = car.WriteBlock(output, block)
if err != nil {
return err
}
recoveredBlocks++
} else {
corruptedBlocks++
log.Printf("유효하지 않은 블록: %s", block.Cid())
}
}
log.Printf("복구 완료: %d 블록 복구됨, %d 블록 손상됨",
recoveredBlocks, corruptedBlocks)
return nil
}


func validateBlock(block blocks.Block) bool {
// 기본 블록 검증
if block == nil {
return false
}
data := block.RawData()
```

```
if len(data) == 0 {
return false
}
// CID 검증
expectedCid, err := block.Cid().Prefix().Sum(data)
if err != nil {
return false
}
return expectedCid.Equals(block.Cid())
}


func skipToNextValidBlock(reader io.Reader) error {
// 간단한 구현: 다음 블록 시작점 찾기
// 실제 구현에서는 더 정교한 복구 로직 필요
buffer := make([]byte, 1024)
_, err := reader.Read(buffer)
return err
}
```

문제 해결

일반적인 문제와 해결책

1. **CAR 파일 손상**

```
// 체크섬 검증 및 복구
if !validateCarFile(carData) {
return  recoverCorruptedCarFile(ctx, reader, output)
}
```

2. **메모리 부족**

```
// 스트리밍 처리 사용
return  streamProcessCarFile(ctx, carReader)
```

3. **네트워크 타임아웃**

```
// 재시도 Fetcher 사용
retryFetcher, _ := gateway.NewRetryCarFetcher(baseFetcher, 5)
```

CAR 파일 처리는 IPFS에서 효율적인 데이터 전송과 아카이브를 위한 핵심 기능입니다. 적절한 최적화와 오류 처리를 통해 안정적이고 효율적인 CAR 파일 시스템을 구축할 수 있습니다.